

Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 30, 2025
Activity 6

Files

<https://github.com/CGalloway3/CST-250-Projects/tree/master/Activity6>

Video

<https://www.loom.com/share/bc2cd3bd543e4dfe88c3ea271fee2a43>

Part 1: Creating the UI

FLOW CHART

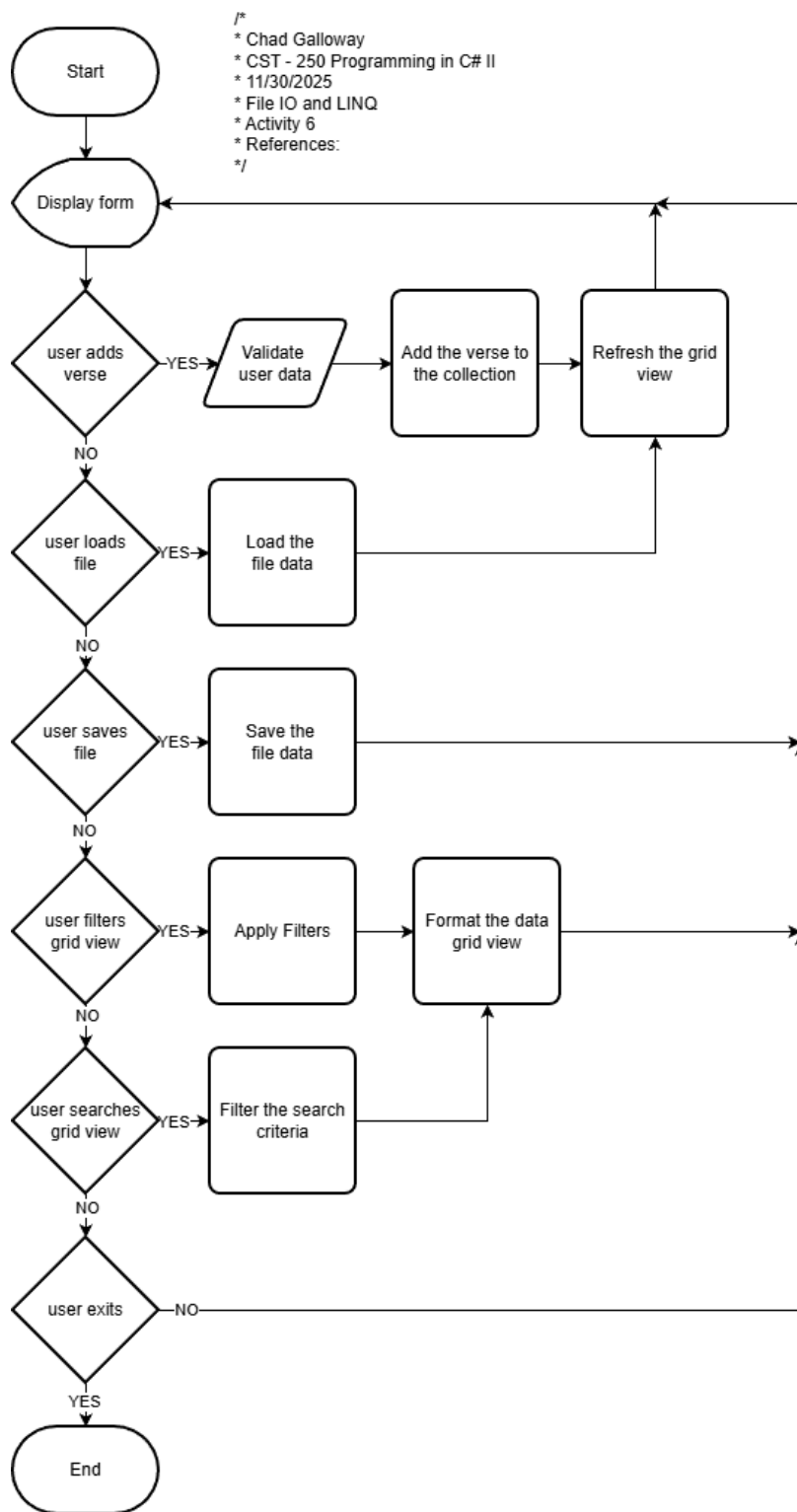


Figure 1: Flow chart of Activity 6

UML Class Diagram

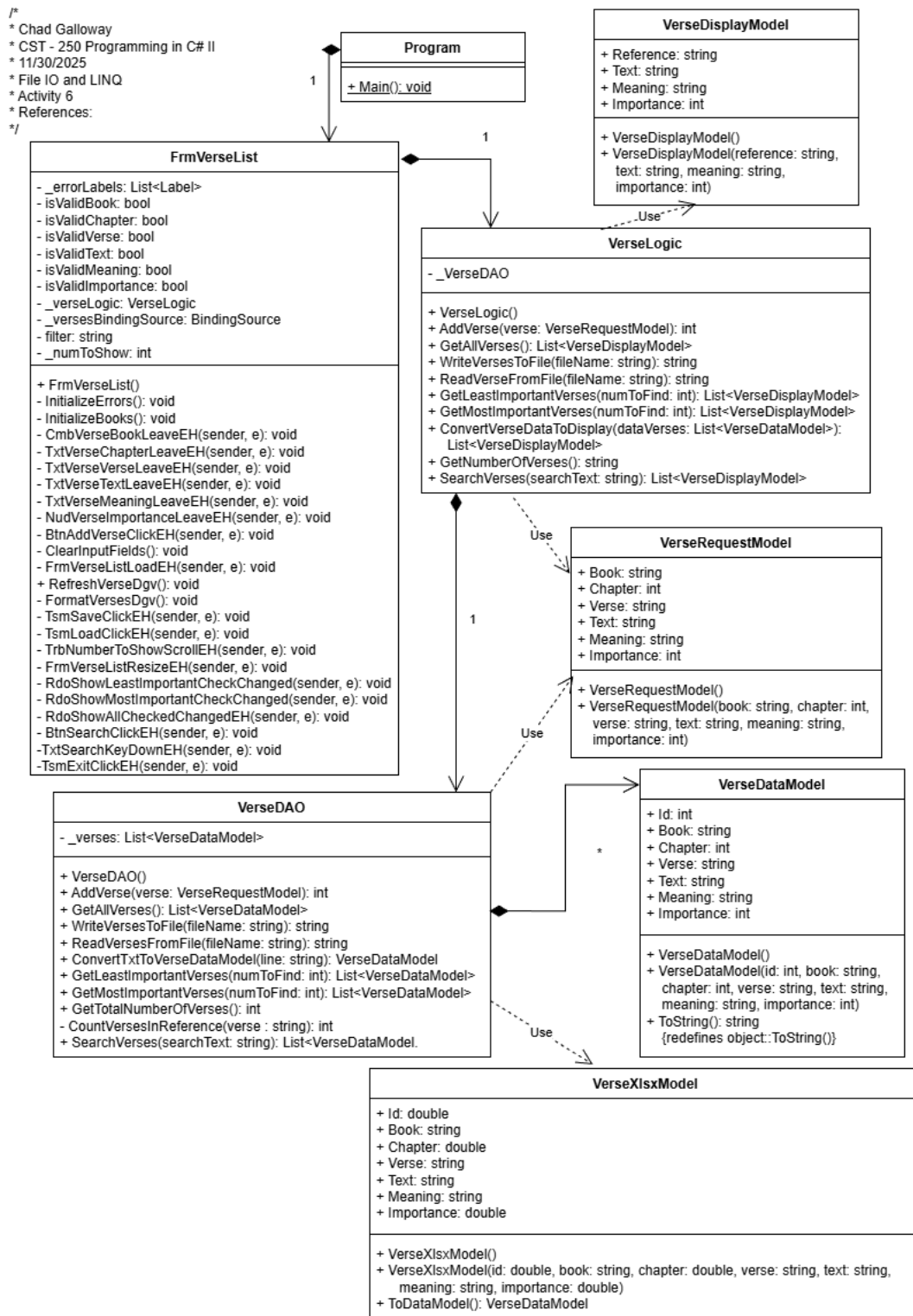


Figure 2: UML Class Diagram

Screen Shots

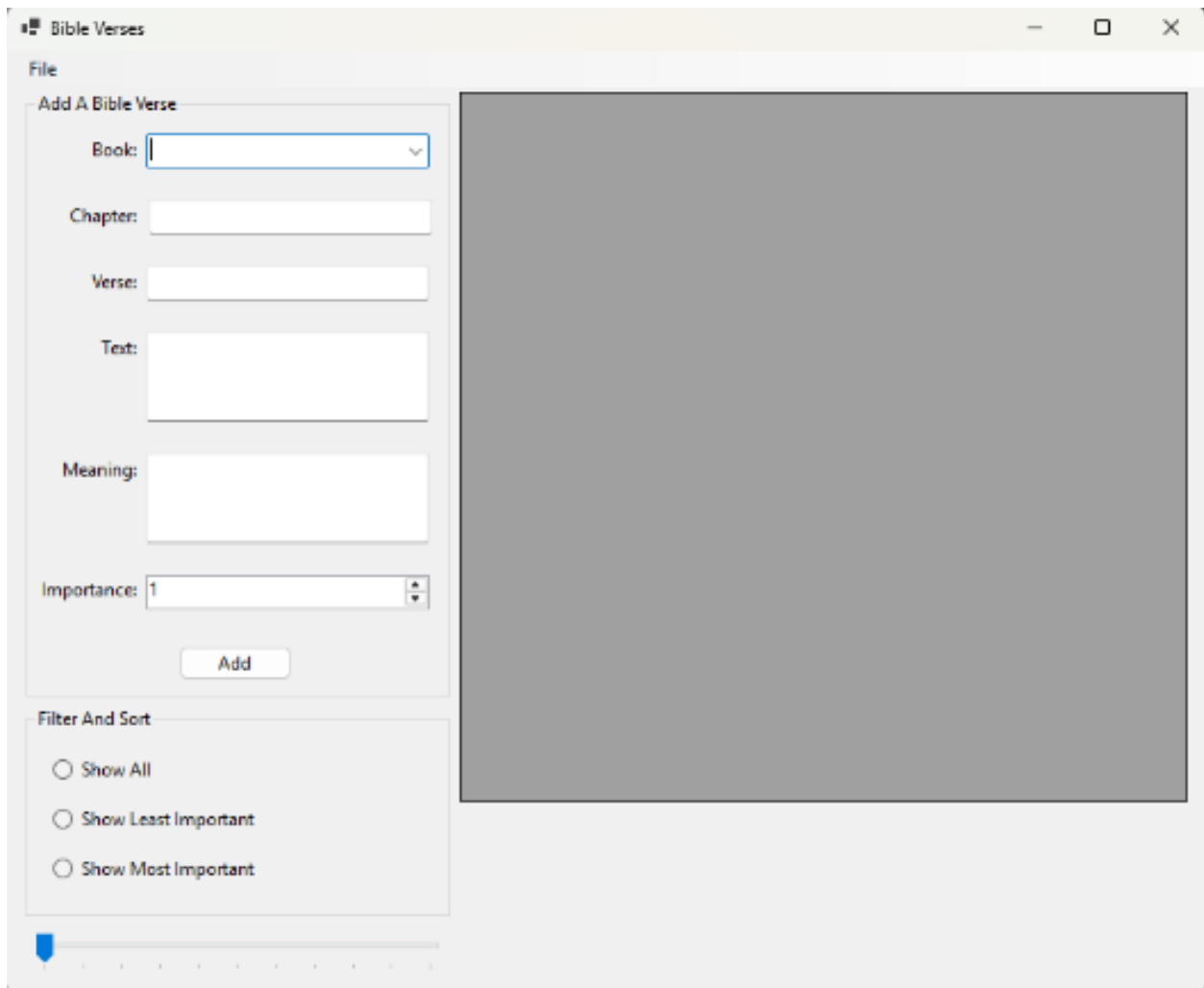


Figure 1-3: Part 1 App Running in the current default load state. This is what the user sees after starting the application.

Bible Verses

File

Add A Bible Verse

Book:

Chapter:

Verse:

Text:

Meaning:

Importance:

1

Add

Filter And Sort

☐ Show All

☐ Show Least Important

☐ Show Most Important

	Reference	Text	Meaning	Importan
▶	Jeremiah 29:11	For I know the plans I have for you,' says the Lord. 'They are plans for good and not for disaster, to give you a future and a hope.'	Jeremiah 29:11 is a promise from God that His plans for humanity are for their good and ultimate well-being, not for harm, and are intended to provide a future with hope	3
*				

Figure 1-4: Screenshot of successful output when the user enters a verse correctly and clicks the add button.

Bible Verses

File

Add A Bible Verse

Book:

You must select a book

Chapter:

The chapter must be a number

Verse:

The verse must be a number/range

Text:

The text cannot be blank

Meaning:

The meaning cannot be blank

Importance:

10

Add

Filter And Sort

Show All

Show Least Important

Show Most Important

	Reference	Text	Meaning	Importan
▶	Jeremiah 29:11	For I know the plans I have for you,' says the Lord. 'They are plans for good and not for disaster, to give you a future and a hope.'	Jeremiah 29:11 is a promise from God that His plans for humanity are for their good and ultimate well-being, not for harm, and are intended to provide a future with hope	3
*				

Figure 1-5: Screenshot of Error Handling the user data entry fields. The NUD control still has an error label but in the Visual Studio IDE I set the minimum and maximum properties to 1 and 10 respectively so the app will not let the user enter values out of that range. If the user were to say enter 55 the number would change automatically to 10. Programmatically changing the value to a number that is out of range causes an error so some of the values on the ClearInputFields() method were adjusted from 0 to 1 to prevent out of bounds errors.

```

1  //
2  * Chad Galloway
3  * CST - 200 Programming in C# II
4  * 11/26/2025
5  * File IO and LINQ
6  * Activity 6
7  * References:
8  //
9
10
11
12 using FileIOAndLINQ.Models;
13 using FileIOAndLINQ.Services.BusinessLogicLayer;
14 using System.Text.RegularExpressions;
15
16 namespace FileIOAndLINQ.PresentationLayer.FrmVerseList
17 {
18     2 references
19     public partial class FrmVerseList : Form
20     {
21         // Declare class level variables
22         private List<Label> _errorLabels;
23         // Flags for user input
24         bool isValidBook = false, isValidChapter = false, isValidVerse = false;
25         bool isValidText = false, isValidMeaning = false, isValidImportance = false;
26         // Business Logic Variable
27         private VerseLogic _verseLogic;
28         // Binding source for the data grid view
29         private BindingSource _versesBindingSource;
30
31         1 reference
32         public FrmVerseList()
33         {
34             InitializeComponent();
35             // Initialize and hide the error list
36             InitializeErrors();
37             // Initialize new verse books
38             InitializeBooks();
39             // Initialize the verse logic variable
40             _verseLogic = new VerseLogic();
41             // Initialize the binding source object
42             _versesBindingSource = new BindingSource();
43         }
44     }

```

Figure 1-6: Screenshot of the FrmVerseList citation, variables, and constructor

```

1  //
2  * Chad Galloway
3  * CST - 200 Programming in C# II
4  * 11/26/2025
5  * File IO and LINQ
6  * Activity 6
7  * References:
8  //
9
10 namespace FileIOAndLINQ.Models
11 {
12     6 references
13     class VerseRequestModel
14     {
15         // Class level properties
16         1 reference
17         public string Book { get; set; }
18         1 reference
19         public int Chapter { get; set; }
20         1 reference
21         public string Verse { get; set; }
22         1 reference
23         public string Text { get; set; }
24         1 reference
25         public string Meaning { get; set; }
26         1 reference
27         public int Importance { get; set; }
28
29         /// <summary>
30         /// Default constructor for the Verse Request Model
31         /// </summary>
32         2 references
33         public VerseRequestModel()
34         {
35             // Set the properties to empty strings
36             Book = string.Empty;
37             Chapter = 0;
38             Verse = string.Empty;
39             Text = string.Empty;
40             Meaning = string.Empty;
41             Importance = 0;
42         }
43
44         /// <summary>
45         /// Parameterized constructor for the Verse Request Model
46         /// </summary>
47         1 reference
48         public VerseRequestModel(string book, int chapter, string verse, string text, string meaning, int importance)
49         {
50             // Set the properties equal to the corresponding parameters
51             Book = book;
52             Chapter = chapter;
53             Verse = verse;
54             Text = text;
55             Meaning = meaning;
56             Importance = importance;
57         }
58     }
59 }

```

Figure 1-7: Screenshot of the VerseRequestModel citation, properties, and constructors.

```
VerseDataModel.cs VerseDisplayModel.cs VerseRequestModel.cs FileIOAndLINQ Models VerseDAO.cs VerseLogic.cs Program.cs
FileIOAndLINQ

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61

// Chad Galloway
// CST - 250 Programming in C# II
// 11/20/2025
// File IO and LINQ
// Activity 6
// References:

namespace FileIOAndLINQ.Models
{
    // 10 references
    class VerseDataModel
    {
        // Class level properties
        // 2 references
        public int Id { get; set; }
        // 1 reference
        public string Book { get; set; }
        // 1 reference
        public int Chapter { get; set; }
        // 1 reference
        public string Verse { get; set; }
        // 1 reference
        public string Text { get; set; }
        // 1 reference
        public string Meaning { get; set; }
        // 1 reference
        public int Importance { get; set; }

        /// <summary>
        /// Default constructor for the Verse Data Model
        /// </summary>
        public VerseDataModel()
        {
            // Set the properties to default values
            Id = 0;
            Book = "";
            Chapter = 0;
            Verse = "";
            Text = "";
            Meaning = "";
            Importance = 0;
        }

        /// <summary>
        /// Parameterized constructor for the Verse Request Model
        /// </summary>
        /// <param name="id"></param>
        /// <param name="book"></param>
        /// <param name="chapter"></param>
        /// <param name="verse"></param>
        /// <param name="text"></param>
        /// <param name="meaning"></param>
        /// <param name="importance"></param>
        public VerseDataModel(int id, string book, int chapter, string verse, string text, string meaning, int importance)
        {
            // Set the properties equal to the corresponding parameters
            Id = id;
            Book = book;
            Chapter = chapter;
            Verse = verse;
            Text = text;
            Meaning = meaning;
            Importance = importance;
        }
    }
}
```

Figure 1-8: Screenshot of the VerseDataModel citation, properties, and constructors.

```
VerseDisplayModel.cs VerseRequestModel.cs FileIOAndLINQ Models VerseDAO.cs VerseLogic.cs VerseDataModel.cs Program.cs
FileIOAndLINQ Reference

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49

// Chad Galloway
// CST - 250 Programming in C# II
// 11/20/2025
// File IO and LINQ
// Activity 6
// References:

namespace FileIOAndLINQ.Models
{
    // 1 reference
    class VerseDisplayModel
    {
        // Class level properties
        // 2 references
        public string Reference { get; set; }
        // 1 reference
        public string Text { get; set; }
        // 1 reference
        public string Meaning { get; set; }
        // 1 reference
        public int Importance { get; set; }

        /// <summary>
        /// Default constructor for the Verse Display Model
        /// </summary>
        public VerseDisplayModel()
        {
            // Set the properties to default values
            Reference = "";
            Text = "";
            Meaning = "";
            Importance = 0;
        }

        /// <summary>
        /// Parameterized constructor for the Verse Display Model
        /// </summary>
        /// <param name="reference"></param>
        /// <param name="text"></param>
        /// <param name="meaning"></param>
        /// <param name="importance"></param>
        public VerseDisplayModel(string reference, string text, string meaning, int importance)
        {
            // Set the properties equal to the corresponding parameters
            Reference = reference;
            Text = text;
            Meaning = meaning;
            Importance = importance;
        }
    }
}
```

Figure 1-9: Screenshot of the VerseDisplayModel citation, properties, and constructors.


```

1  //
2  // Chad Galloway
3  // CST - 280 Programming in C# II
4  // 11/28/2025
5  // File 10 and LINQ
6  // Activity 6
7  // References:
8  //
9
10 using FileIOAndLINQ.Models;
11 using FileIOAndLINQ.Services.DataAccessLayer;
12
13 namespace FileIOAndLINQ.Services.BusinessLogicLayer
14 {
15     3 references
16     class VerseLogic
17     {
18         // Class level variables
19         private VerseDAO _verseDAO;
20
21         /// <summary>
22         /// Default constructor for VerseLogic
23         /// </summary>
24         public VerseLogic()
25         {
26             // Initialize the data access object
27             _verseDAO = new VerseDAO();
28         }
29
30         1 reference
31         public int AddVerse(VerseRequestModel verse)
32         {
33             // return the DAO method
34             return _verseDAO.AddVerse(verse);
35         }
36
37         /// <summary>
38         /// Get a list of verses from the inventory
39         /// </summary>
40         /// <returns>~/returns>
41         1 reference
42         public List<VerseDisplayModel> GetAllVerses()
43         {
44             // Declare and initialize
45             // Get the verses from the DAO
46             List<VerseDataModel> dataVerses = _verseDAO.GetAllVerses();
47             // Create a DisplayModel list
48             List<VerseDisplayModel> displayVerses = new List<VerseDisplayModel>();
49             string reference = "";
50
51             // Loop through the dataVerses list
52             foreach (VerseDataModel verse in dataVerses)
53             {
54                 // Use the book, chapter, and verse to create the reference
55                 reference = $"{verse.Book} {verse.Chapter} {verse.Verse}";
56                 // Create a display verse model using the VerseDataModel verse
57                 VerseDisplayModel displayVerse = new VerseDisplayModel(reference, verse.Text, verse.Meaning, verse.Importance);
58                 // Add the display model to the displayVerses list
59                 displayVerses.Add(displayVerse);
60             }
61
62             // Return the display verses list
63             return displayVerses;
64         }
65     }
66 }

```

Figure 1-10: Screenshot of the VerseLogic citation, variables, constructor, and methods AddVerse() and GetAllVerses()

```

1  //
2  // Chad Galloway
3  // CST - 280 Programming in C# II
4  // 11/28/2025
5  // File 10 and LINQ
6  // Activity 6
7  // References:
8  //
9
10 using FileIOAndLINQ.Models;
11
12 namespace FileIOAndLINQ.Services.DataAccessLayer
13 {
14     2 references
15     class VerseDAO
16     {
17         // Declare class level variables
18         List<VerseDataModel> _verses;
19
20         /// <summary>
21         /// Default constructor for VerseDAO
22         /// </summary>
23         public VerseDAO()
24         {
25             // Create a new list of VerseDataModel
26             _verses = new List<VerseDataModel>();
27         }
28
29         /// <summary>
30         /// Add a new verse to the inventory
31         /// </summary>
32         /// <param name="verse">~/param>
33         /// <returns>~/returns>
34         1 reference
35         public int AddVerse(VerseRequestModel verse)
36         {
37             // Declare and initialize
38             int id = _verses.Count + 1;
39             VerseDataModel newVerse = new VerseDataModel(id);
40
41             // Create a new verse based on the verse request model
42             newVerse = new VerseDataModel(id, verse.Book, verse.Chapter,
43             verse.Verse, verse.Text, verse.Meaning, verse.Importance);
44             // Add the new verse to the verses list
45             _verses.Add(newVerse);
46
47             // Return the id of the new verse
48             return id;
49         }
50
51         1 reference
52         public List<VerseDataModel> GetAllVerses()
53         {
54             // Return the _verses list
55             return _verses;
56         }
57     }
58 }

```

Figure 1-11: Screenshot of the VerseDAO citation, variables, constructor, and methods AddVerse() and GetAllVerses()

```

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/30/2025
 * File IO and LINQ
 * Activity 6
 * References:
 */

```

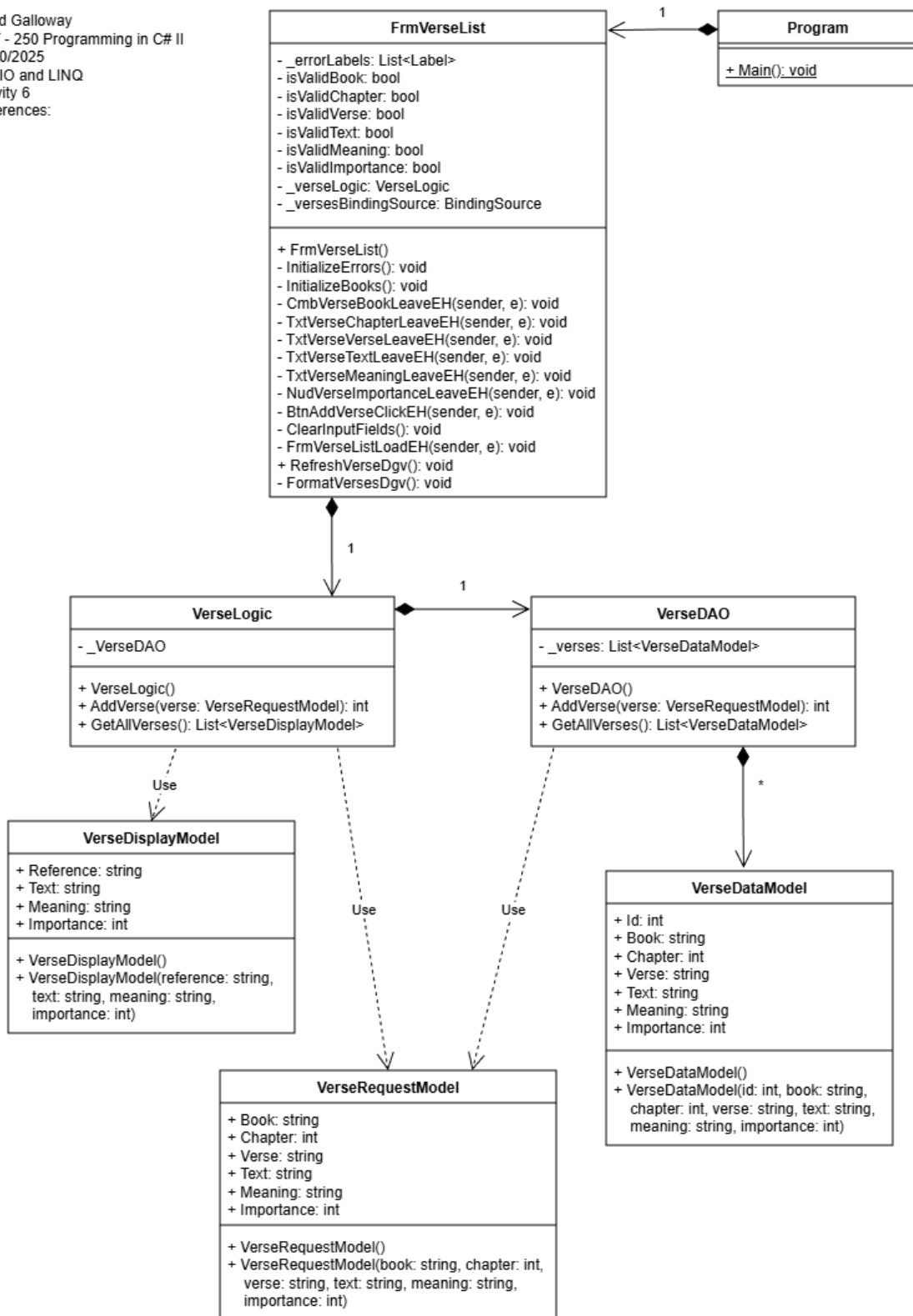


Figure 1-12: Screenshot of the UML for all the classes as they are now.

Part 2: File I/O

Screen Shots

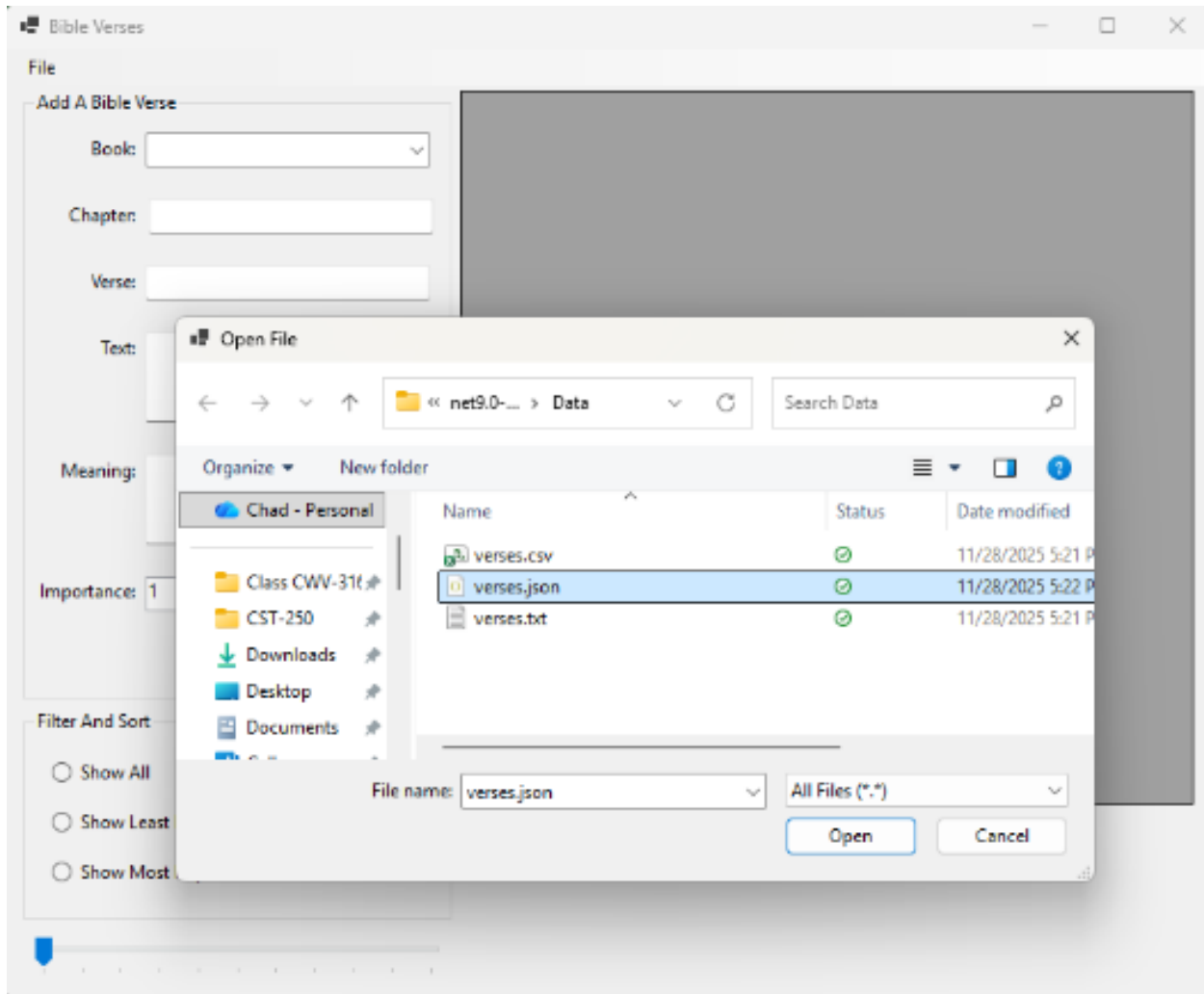


Figure 2-1: Part 2 Application load file dialog.

Bible Verses

File

Add A Bible Verse

Book:

Chapter:

Verse:

Text:

Meaning:

Importance:

1

Add

Filter And Sort

☐ Show All

☐ Show Least Important

☐ Show Most Important

	Reference	Text	Meaning	Importan
▶	Proverbs 22:6	Train a boy in the way he should go; even when he grows old he will not depart from it.	Parents who teach their children to love God and respect his laws can expect such training to have a positive and lasting influence.	4
	Isaiah 42:8	I am Jehovah. That is my name; I give my glory to no one else, nor my praise to graven images.	God tells us his personal name and adds that he does not share his praise or honor with idols.	2
	John 3:16	For God loved the world so much that he gave his only-begotten Son, so that everyone exercising faith in him might not be destroyed but have everlasting life.	God loves us and wants us to live forever. For that reason, he sent his Son, Jesus Christ, to the earth. While on earth, Jesus accomplished much. For one thing, he taught his followers about his God and Father. (1 Peter 1:3) For another, he gave his life for mankind. To gain everlasting life, we must have faith in Jesus.	9
*				

Figure 2-2: Screenshot of successful file loaded.

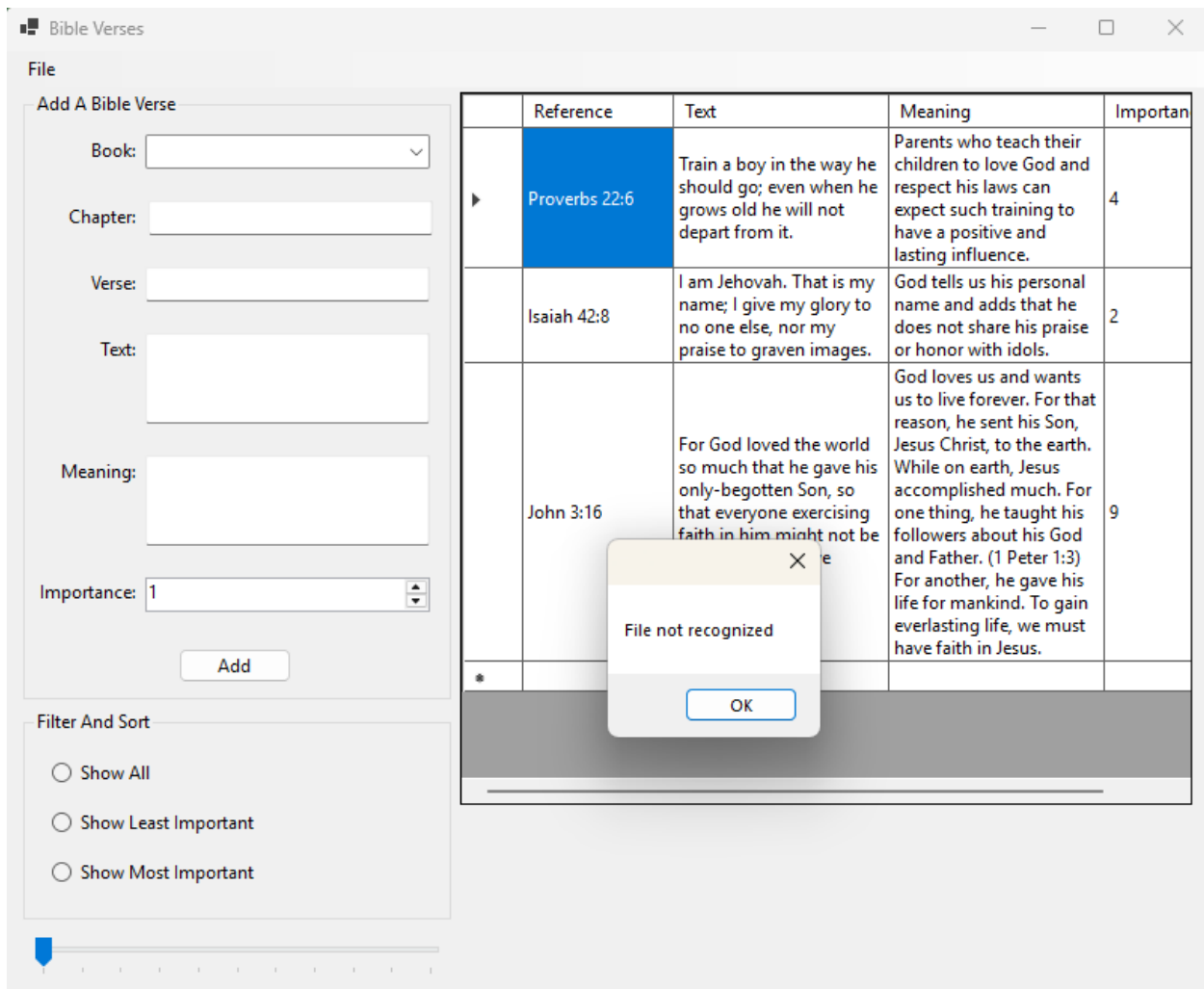


Figure 2-3: Screenshot of Error Handling for invalid file types.

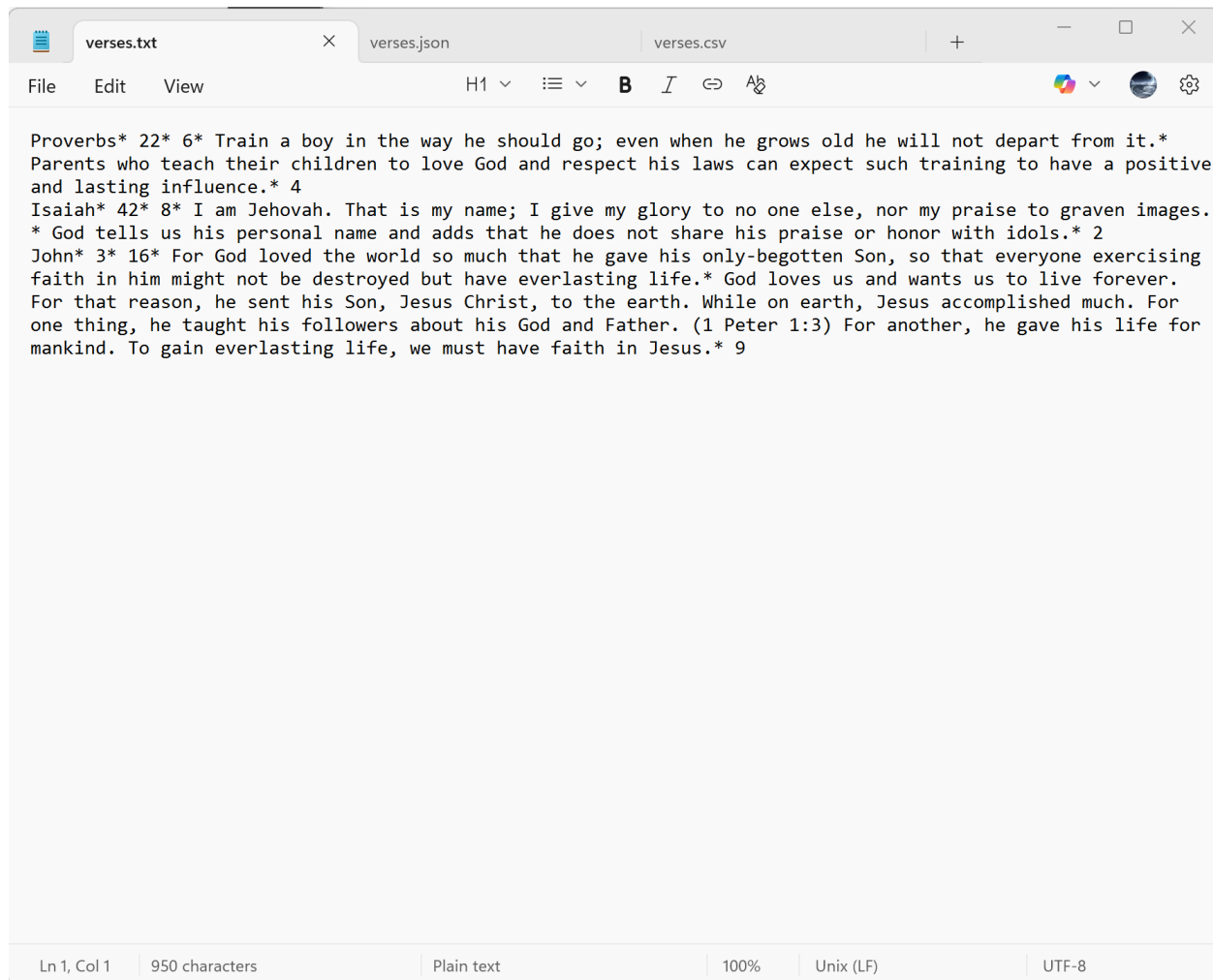


Figure 2-4: Screenshot of .txt verses file

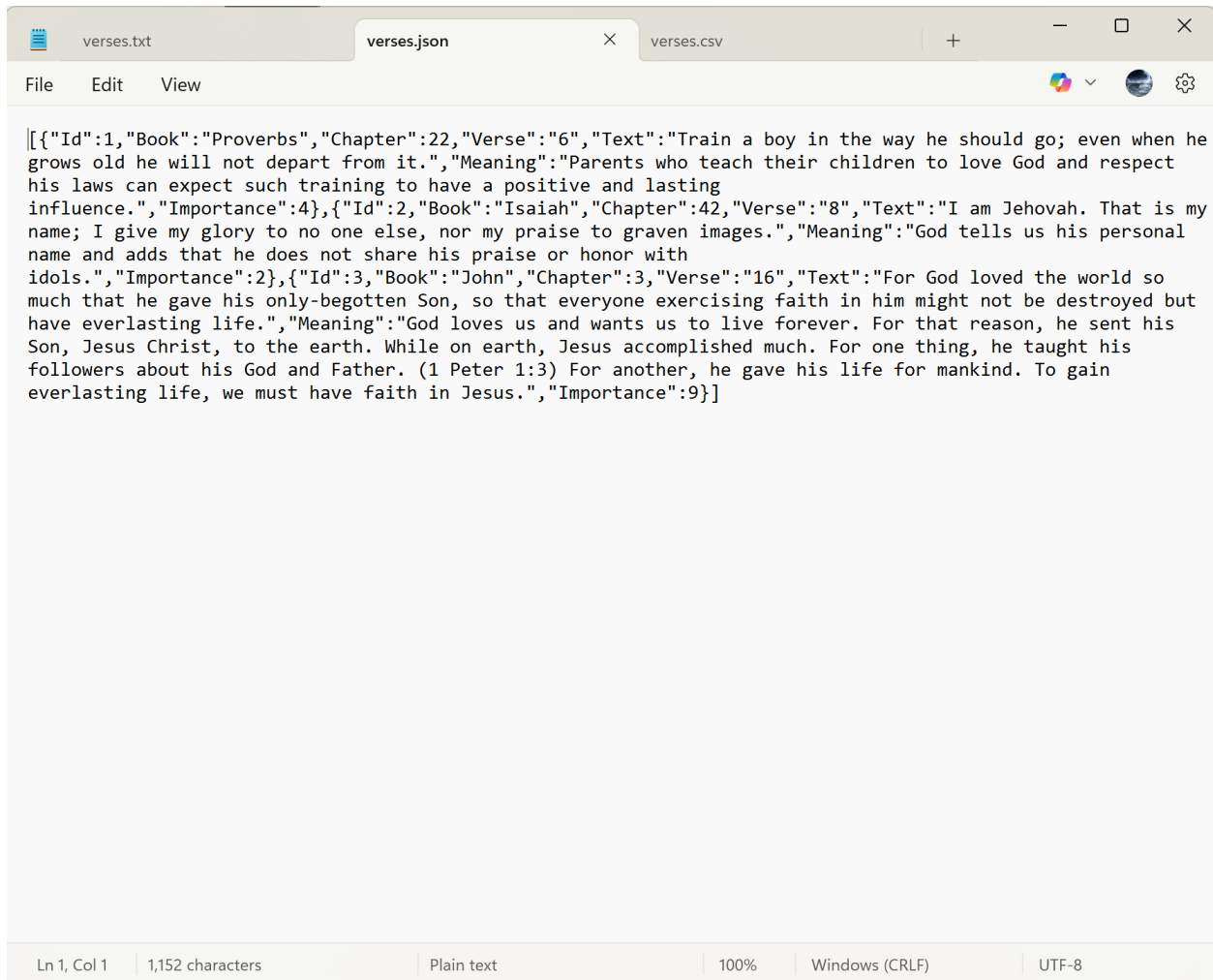


Figure 2-5: Screenshot of .json verses file

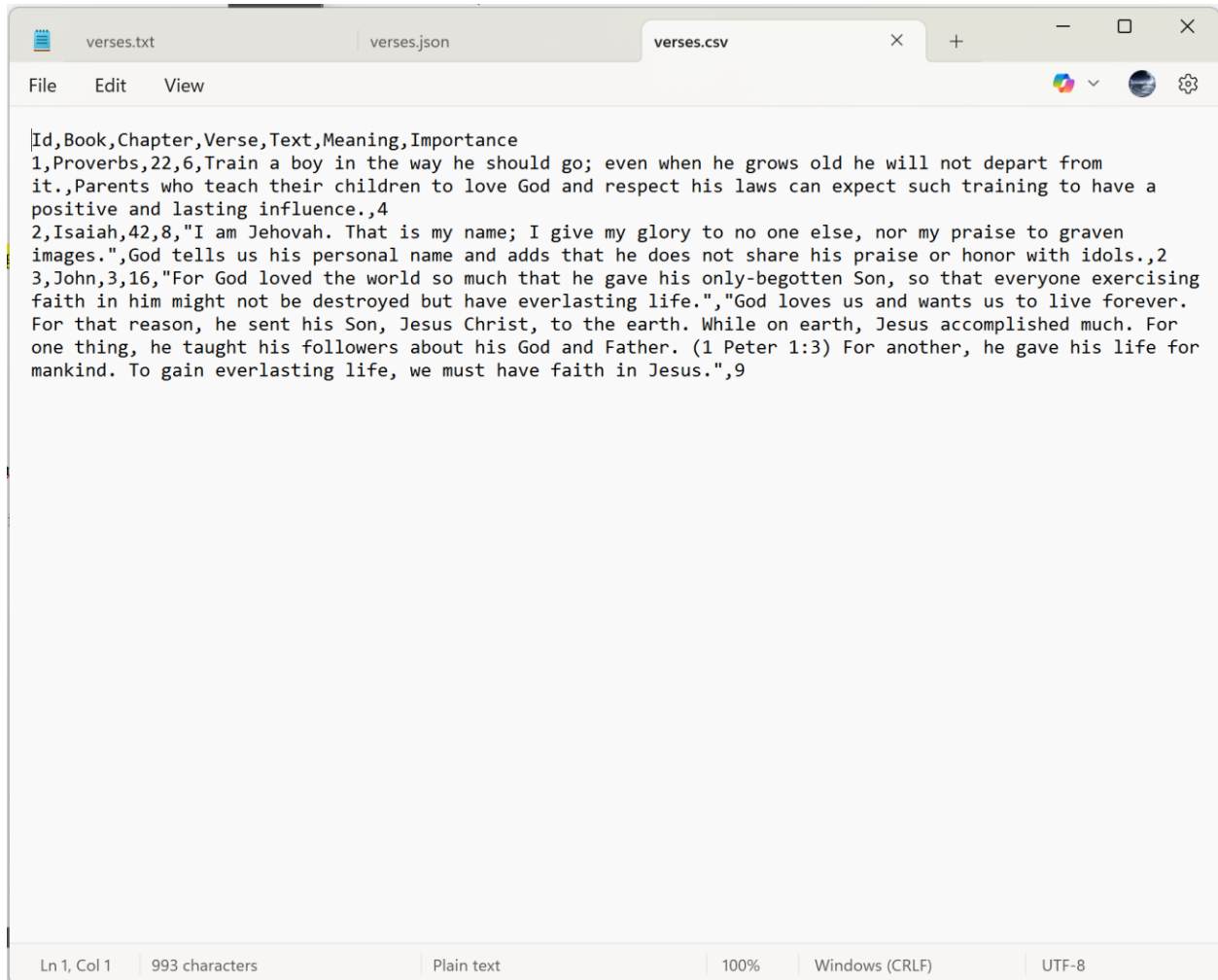


Figure 2-6: Screenshot of .csv verses file

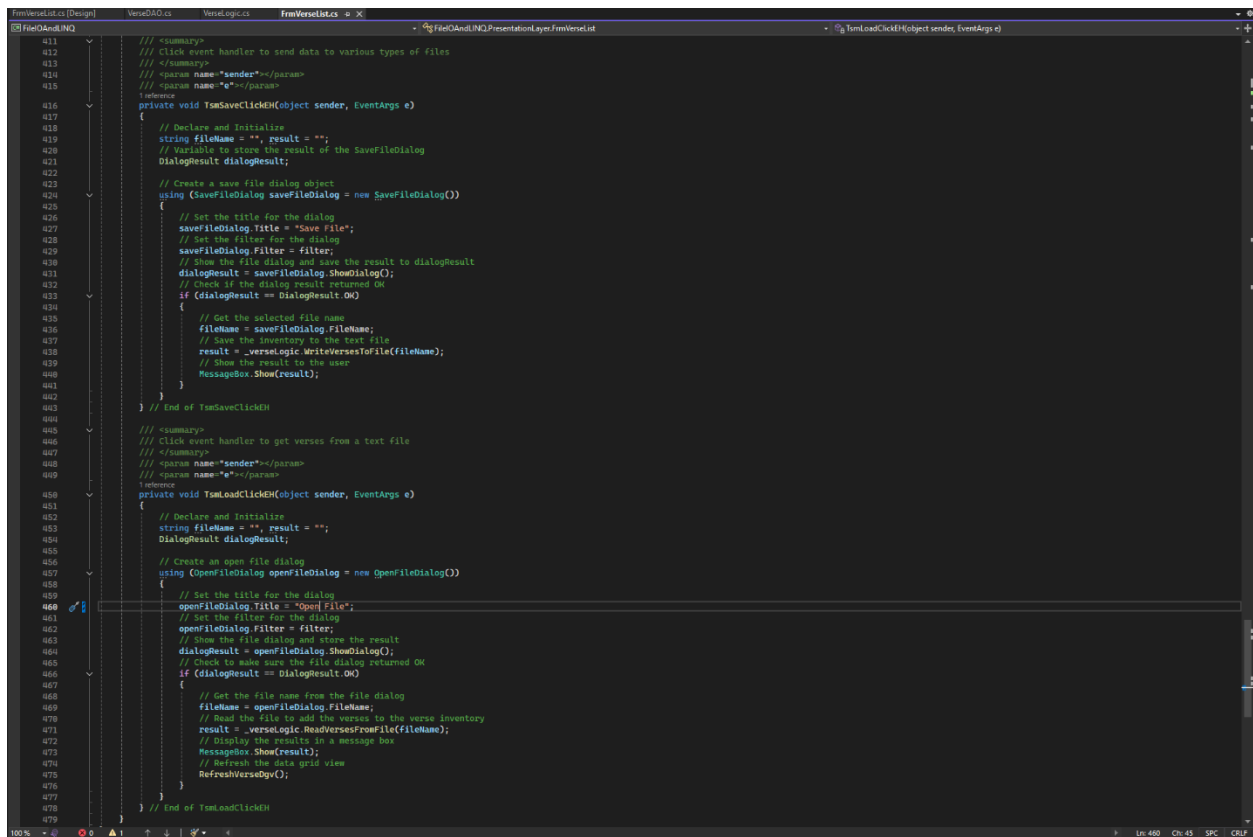


Figure 2-7: Screenshot of the save and load click event handlers for the menu items save and load.

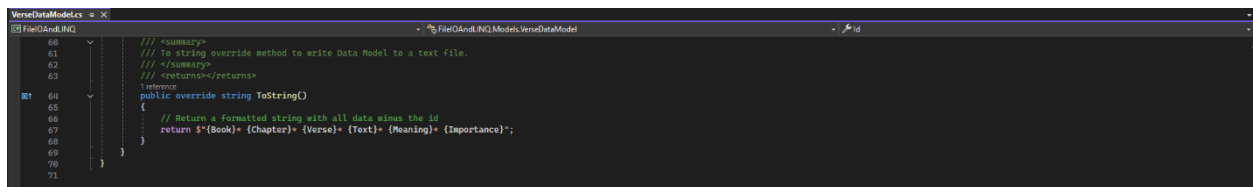


Figure 2-8: Screenshot of the overridden ToString() method within the VerseDataModel

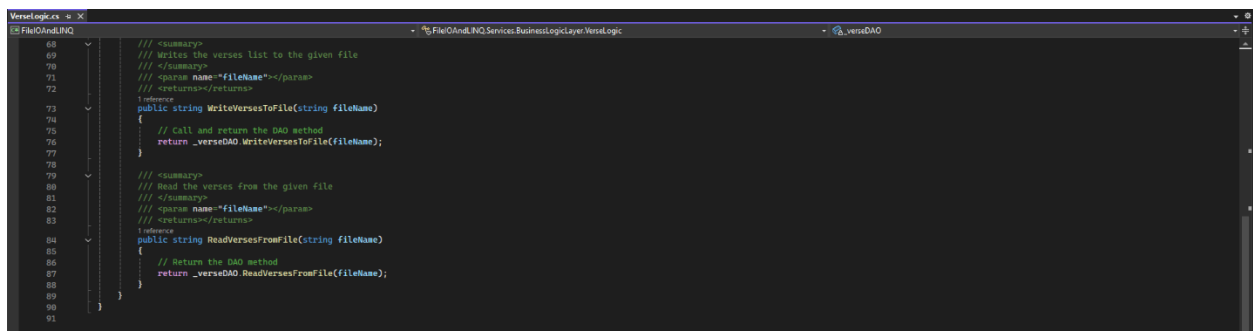


Figure 2-9: Screenshot of the write verses to file and the read verses from file methods inside the verse logic class.

```

VerseDAO.cs
FileDAOAndLINQ
// <summary>
// Writes the verse list to the given file
// formatted based on file type
// </summary>
// <param name="fileName"></param>
// <returns></returns>
// </param>
public string WriteVersesToFile(string fileName)
{
    // Declare and initialize
    string serialized = "";

    // Create a switch based on file extensions to fill the serialized string
    switch (Path.GetExtension(fileName))
    {
        case ".txt":
            // Loop through the _verses list
            foreach (var verse in _verses)
            {
                // Add each verse to the end of the serialized string
                serialized += verse.ToString() + "\n";
            }
            break;
        case ".json":
            // Use ServiceStack to serialize to json
            serialized = ServiceStack.Text.JsonSerializer.SerializeToString(_verses);
            break;
        case ".csv":
            // Use ServiceStack to serialize to csv
            serialized = ServiceStack.Text.CsvSerializer.SerializeToString(_verses);
            break;
        default:
            return "File not recognized";
    }

    // Try to save the serialized string to file.
    try
    {
        // Use File.WriteAllText to send the serialized string to the file
        File.WriteAllText(fileName, serialized);
    }
    catch (Exception ex)
    {
        return ex.Message;
    }

    // Return success message to the user
    return "The verses have been saved to your file.";
}

```

Figure 2-10: Screenshot of the write verses to file method inside the verse DAO class.

```

VerseDAO.cs
FileDAOAndLINQ
// <summary>
// Read verses from the given file
// </summary>
// <param name="fileName"></param>
// <returns></returns>
// </param>
public string ReadVersesFromFile(string fileName)
{
    // Declare and initialize
    string data = "";
    List<VerseDataModel> dataVerses = new List<VerseDataModel>();

    // Set up a try-catch to read files text
    try
    {
        // Get the text from the file
        data = File.ReadAllText(fileName);
    }
    catch (Exception ex)
    {
        // Return the exception message
        return ex.Message;
    }

    // Create a switch based on the file extension
    switch (Path.GetExtension(fileName))
    {
        case ".txt":
            // Split the text file on the newline character
            string[] lines = data.Split('\n');
            // Loop through the array of lines
            foreach (string line in lines)
            {
                // Check if each line contains data
                if (!string.IsNullOrEmpty(line))
                {
                    // If so, convert the data to a VerseDataModel
                    // and add it to the dataVerses list
                    dataVerses.Add(ConvertTextToVerseDataModel(line));
                }
            }
            break;
        case ".json":
            // Deserialize the data using the JsonSerializer
            dataVerses = ServiceStack.Text.JsonSerializer.DeserializeFromJson<List<VerseDataModel>>(data);
            break;
        case ".csv":
            // Deserialize the data using the CsvSerializer
            dataVerses = ServiceStack.Text.CsvSerializer.DeserializeFromCsv<List<VerseDataModel>>(data);
            break;
        default:
            // Return the issue to the user
            return "File not recognized";
    }

    // Loop through the dataVerses list
    foreach (VerseDataModel newVerse in dataVerses)
    {
        // Set the id for each new verse
        newVerse.Id = _verses.Count + 1;
        // Add the new verse to the _verses list
        _verses.Add(newVerse);
    }

    // Return a success message to the user
    return "The verses have been read from your file and added to the list";
}
// End of ReadVersesFromFile

```

Figure 2-11: Screenshot of the read verses from file method inside the verse DAO class.

```
VerseDAO.cs x
FileDAOAndLINQ Services.DataAccessLayer.VerseDAO
G:\_verses

174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203

/// <summary>
/// Take a line from the text file and return a VerseDataModel
/// </summary>
/// <param name="line">~/param>
/// <returns>~/returns>
/// </returns>
/// </summary>
public VerseDataModel ConvertTxtToVerseDataModel(string line)
{
    // Declare and initialize
    string[] values;
    int chapter = 0; importance = 0;
    VerseDataModel verse;

    // Split the line on ' '
    values = line.Split(' ');

    // Use a try parse to parse the chapter
    int.TryParse(values[1], out chapter);

    // Parse the importance
    int.TryParse(values[3], out importance);

    // Create the new verse
    verse = new VerseDataModel(0, values[0], chapter, values[2], values[3], values[4], importance);

    // Return the verse
    return verse;
}
```

Figure 2-12: Screenshot of the convert txt to verse data model

```

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/30/2025
 * File IO and LINQ
 * Activity 6
 * References:
 */

```

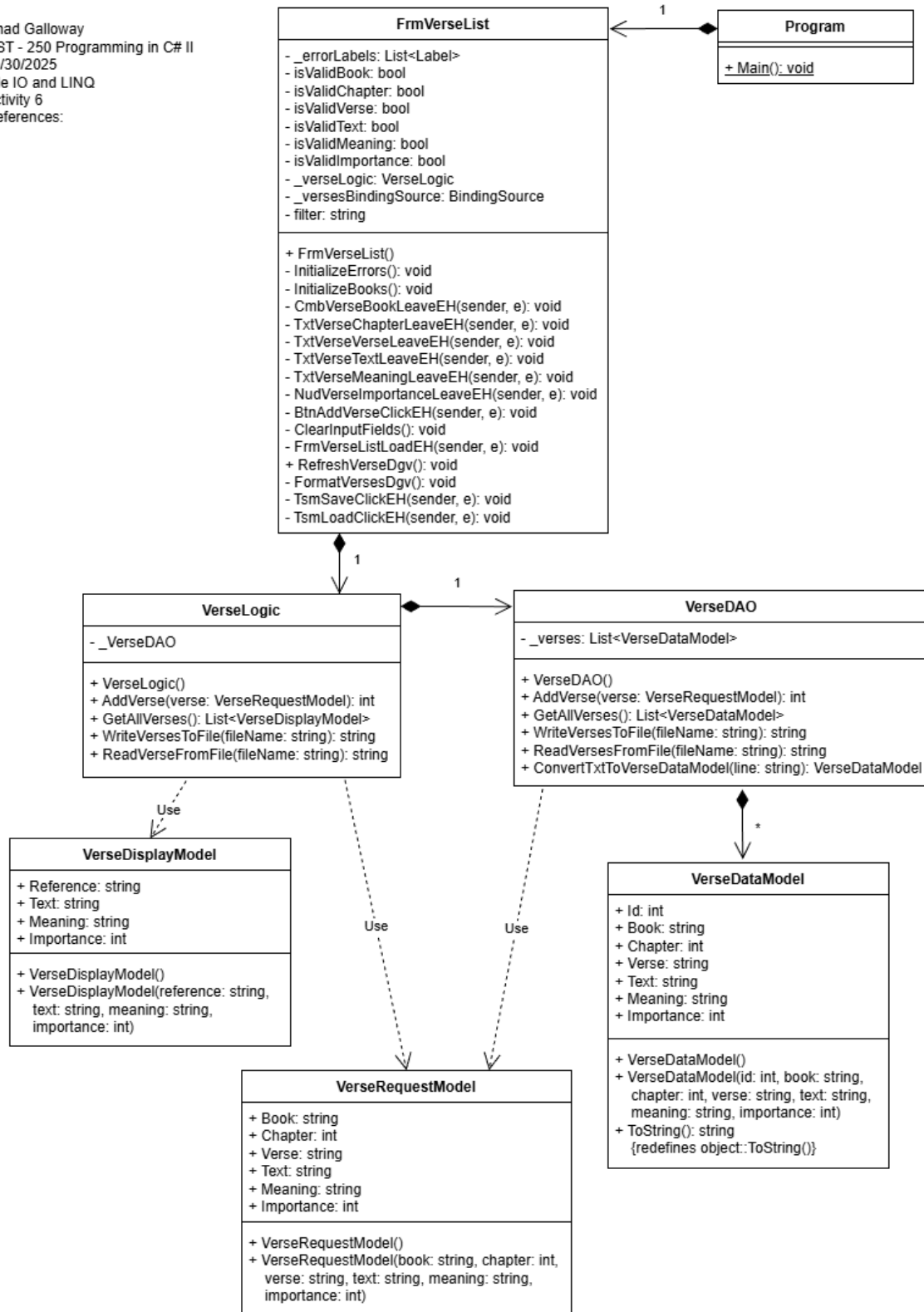


Figure 2-13: Screenshot of UML as the project exists now.

Part 3: Filter and Sort

Screen Shots

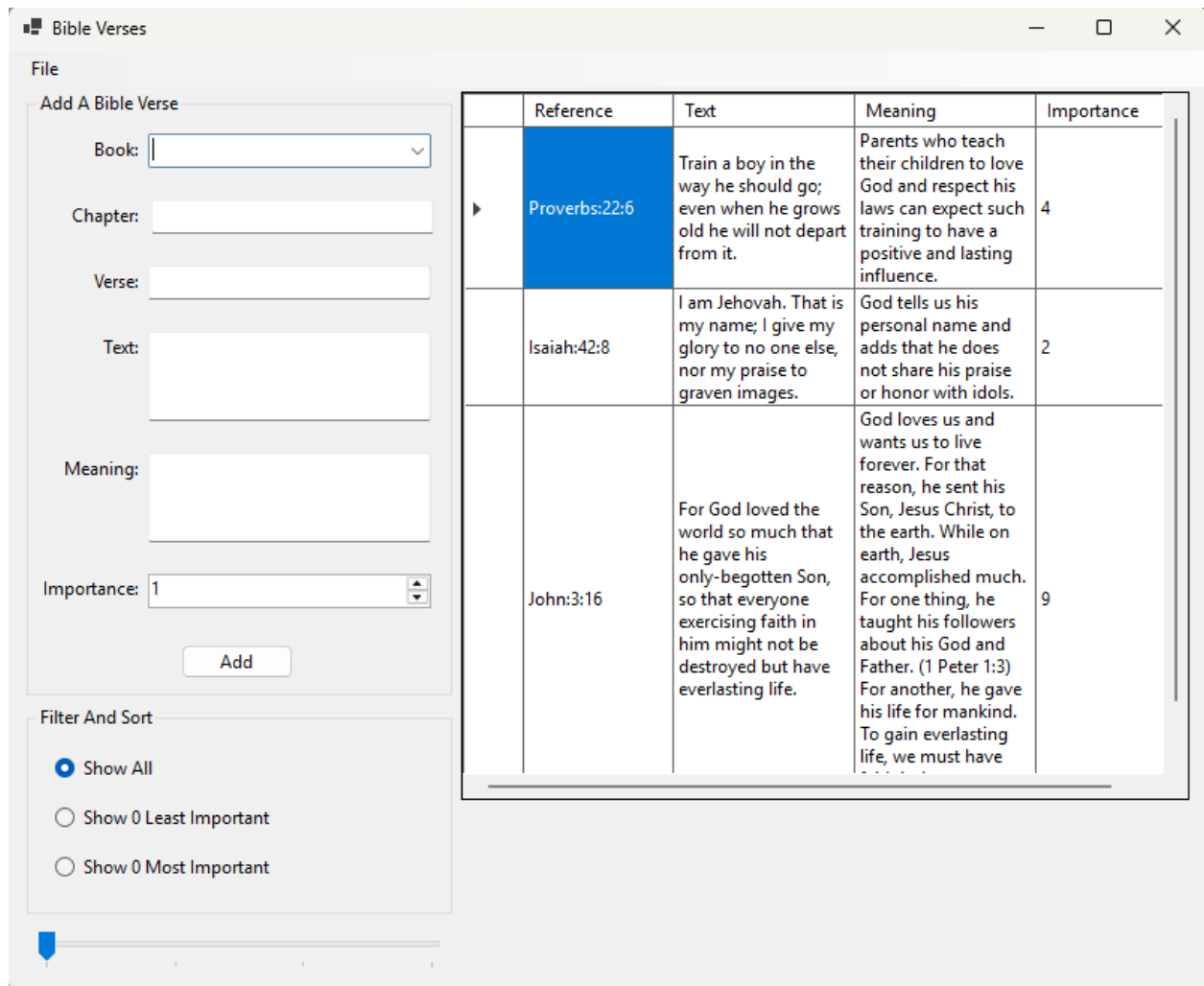


Figure 3-1: Part 3 App Running with a file loaded in. I set the application to start with the Show All radio button checked and the text for the other two radio buttons to display the number 0 as this better represents the starting state of the application to the user.

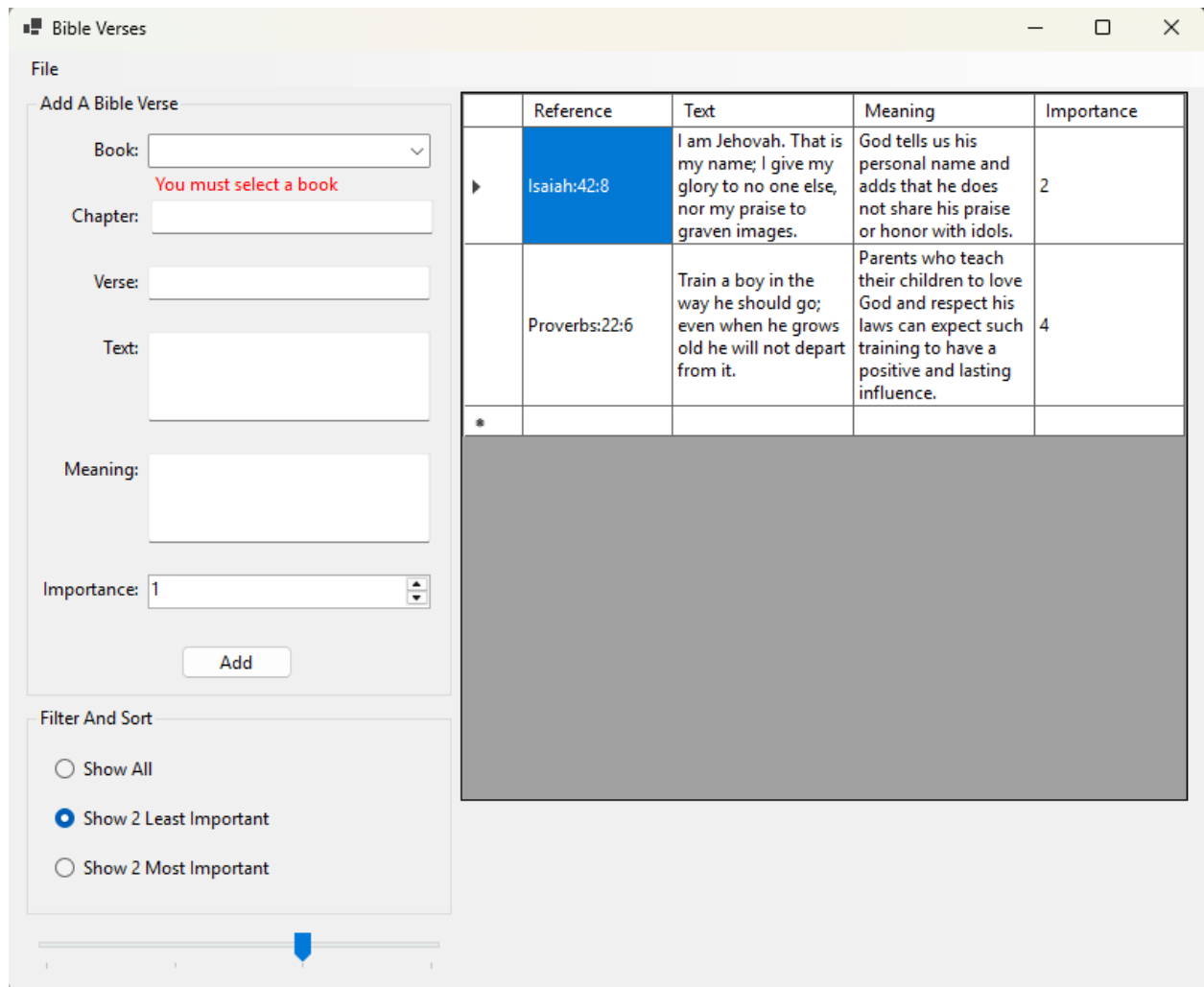


Figure 3-2: Screenshot of the application showing only the least important verses. I added handling of the adjustment of the track bar while a filter is selected by calling the selected changed event handlers when the scroll changes because previously the user had to change filters for track bar scroll changes to be visible.

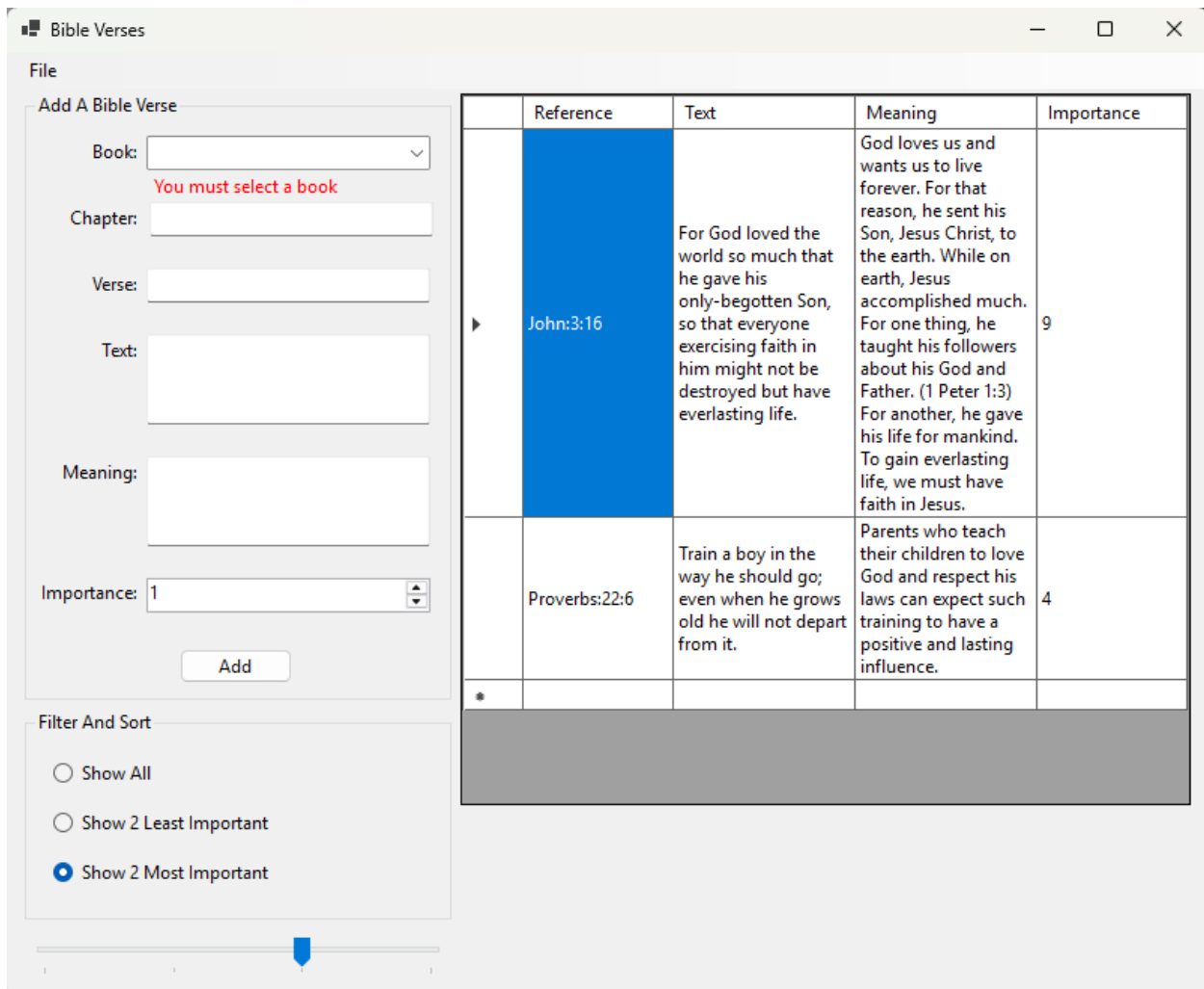


Figure 3-3: Screenshot of the application showing only the most important verses.

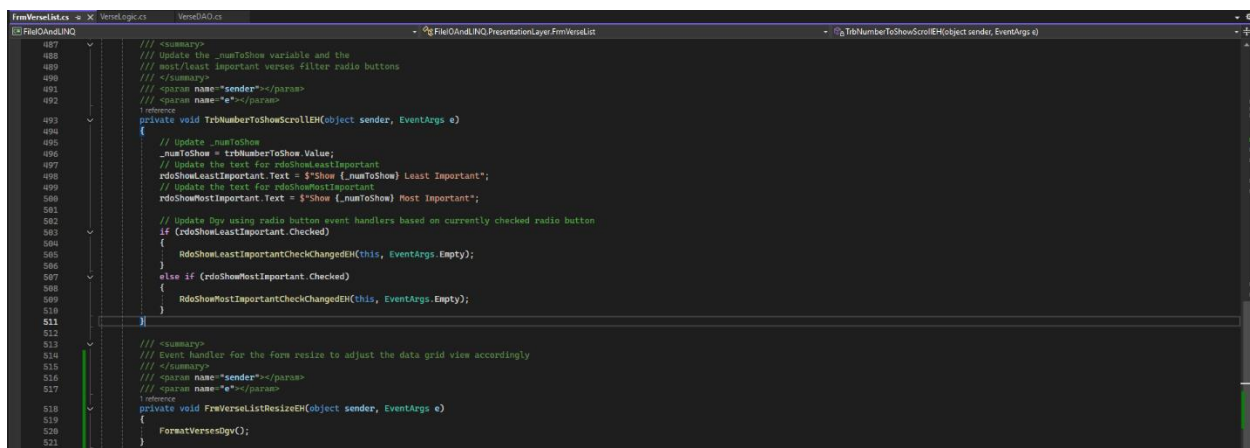
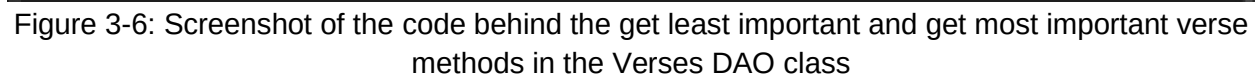
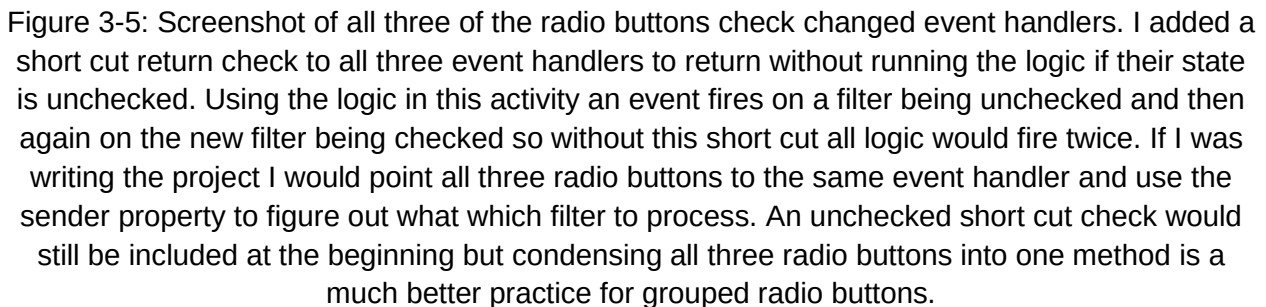


Figure 3-4: Screenshot of the track bar number to show scroll event handler and an additional form resize event handler I added in to keep the DGV the correct size if the user resizes the form. I also added a few lines in the track bar scroll event handler to recall the event handlers for the radio buttons to keep the DGV up to date when the track bar number is changed.




```

76  /// <summary>
77  /// Get a list of the least important verses
78  /// </summary>
79  /// <param name="numToFind"></param>
80  /// <returns></returns>
81  /// reference
82  public List<VerseDisplayModel> GetLeastImportantVerses(int numToFind)
83  {
84      // Get a list of the least important data verses
85      List<VerseDataModel> dataVerses = _verseDAO.GetLeastImportantVerses(numToFind);
86      // Convert the dataVerses list to a displayVersesList and return
87      return ConvertVerseDataToDisplay(dataVerses);
88  }
89
90  /// <summary>
91  /// Get a list of the most important verses
92  /// </summary>
93  /// <param name="numToFind"></param>
94  /// <returns></returns>
95  /// reference
96  public List<VerseDisplayModel> GetMostImportantVerses(int numToFind)
97  {
98      // Get a list of the least important data verses
99      List<VerseDataModel> dataVerses = _verseDAO.GetMostImportantVerses(numToFind);
100     // Convert the dataVerses list to a displayVersesList and return
101     return ConvertVerseDataToDisplay(dataVerses);
102 }
103
104 /// <summary>
105 /// Convert a list of VerseDataModels to VerseDisplayModels
106 /// </summary>
107 /// <param name="dataVerses"></param>
108 /// <returns></returns>
109 /// reference
110 public List<VerseDisplayModel> ConvertVerseDataToDisplay(List<VerseDataModel> dataVerses)
111 {
112     // Declare and initialize
113     List<VerseDisplayModel> displayVerses = new List<VerseDisplayModel>();
114     string reference = "";
115
116     // Loop through the dataVerses list
117     foreach (VerseDataModel verse in dataVerses)
118     {
119         // Use the book, chapter, and verse to create the reference
120         reference = $"{verse.Book}:{verse.Chapter}:{verse.Verses}";
121
122         // Create a display verse model using the VerseDataModel verse
123         VerseDisplayModel displayVerse = new VerseDisplayModel(reference, verse.Text, verse.Meaning, verse.Importance);
124
125         // Add the display model to the displayVerses list
126         displayVerses.Add(displayVerse);
127     }
128
129     // Return the display verses list
130     return displayVerses;
131 }

```

Figure 3-7: Screenshot of the get most important verse and get least important verse methods from the verse logic class along with the convert verse data to display from the same class that both methods use for converting their input parameter(dataModel) to displayable return(displayModel) values.

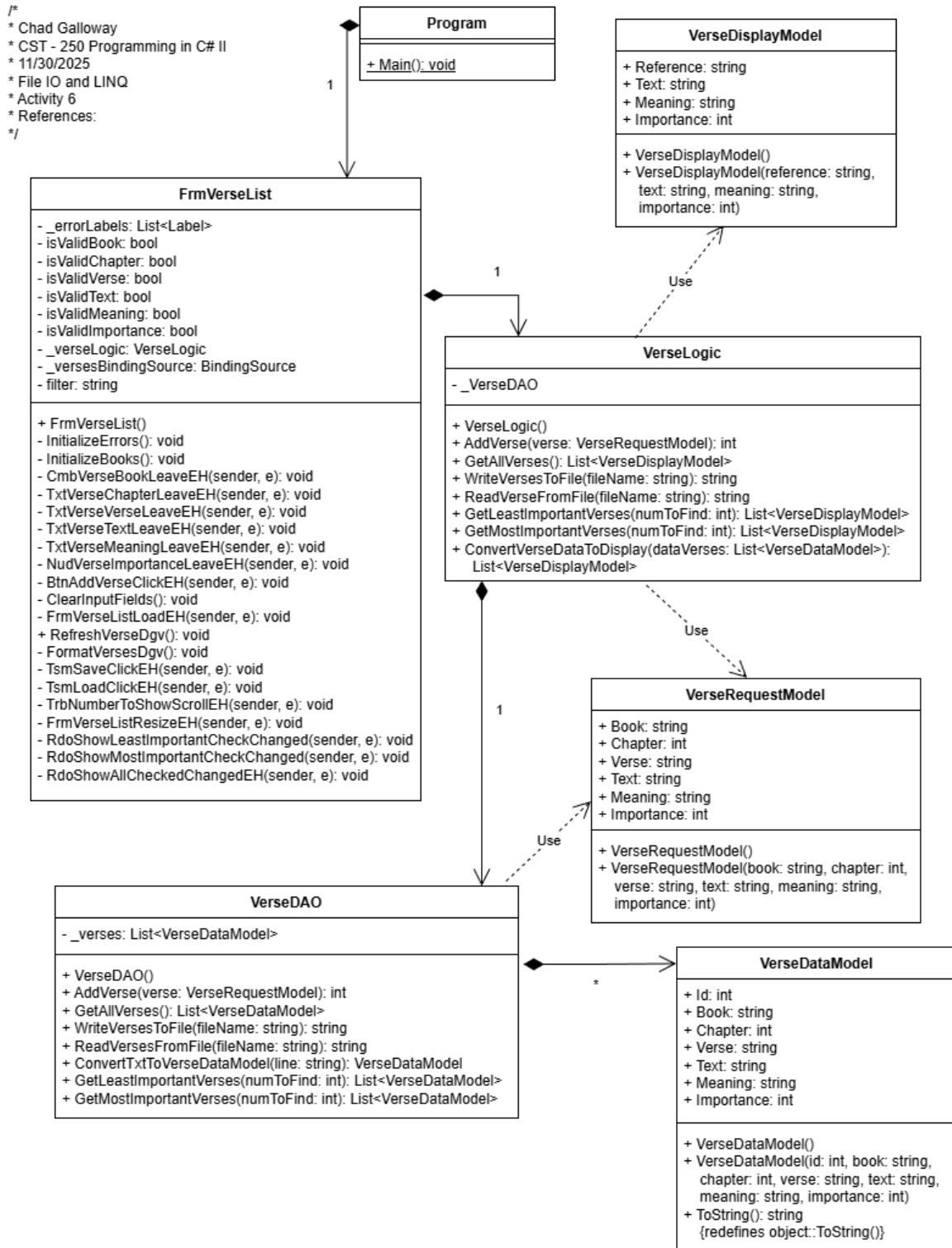


Figure 3-8: Screen shot of the UML as the project exists currently.

Part 4: Challenges

Screen Shots

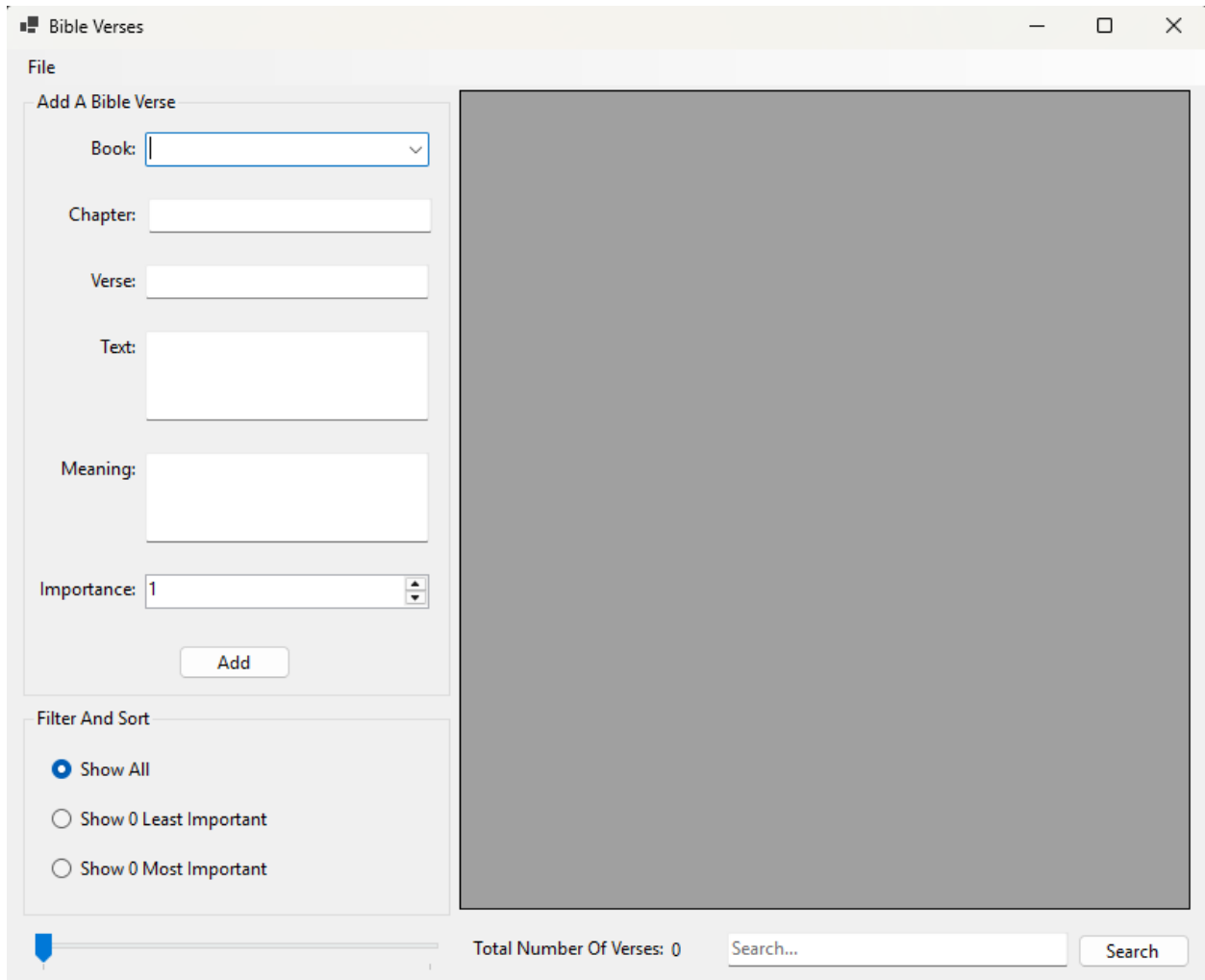


Figure 4-1: Part 4 App Running in the initial load state. This is what the user will see when they first launch the application.

Bible Verses

File

Add A Bible Verse

Book:

Chapter:

Verse:

Text:

Meaning:

Importance:

1

Add

Filter And Sort

☒ Show All

☐ Show 0 Least Important

☐ Show 0 Most Important

Reference	Text	Meaning	Importance
John 8:32	Then you will know the truth, and the truth will set you free.	The meaning is that a spiritual, experiential knowledge of God's truth, particularly through following Jesus, leads to freedom from the bondage of sin, guilt, and deception.	7
Proverbs 22:6	Train a boy in the way he should go; even when he grows old he will not depart from it.	Parents who teach their children to love God and respect his laws can expect such training to have a positive and lasting influence.	4
Isaiah 42:8	I am Jehovah. That is my name; I give my glory to no one else, nor my praise to graven images.	God tells us his personal name and adds that he does not share his praise or honor with idols.	2
John 3:16	For God loved the world so much that he gave his only-begotten Son, so that everyone exercising faith in him might not be destroyed but have everlasting life.	God loves us and wants us to live forever. For that reason, he sent his Son, Jesus Christ, to the earth. While on earth, Jesus accomplished much. For one thing, he taught his followers about his God and Father. (1 Peter 1:3) For another, he gave his life for mankind. To gain everlasting life, we must have faith in Jesus.	9
2 Corinthians 4:16-18	So we do not lose heart. Though our outer self is wasting away, our inner self is being renewed day by day. For this light momentary affliction is preparing for us an eternal weight of glory beyond all comparison, as we look not to the things that are seen but to the things that are unseen.	The meaning is to shift focus from the temporary, physical, and seen to the eternal, spiritual, and unseen, which provides hope and strength for daily renewal.	3
*			

Total Number Of Verses: 7

Search...

Search

Figure 4-2: Screenshot of successful file loaded in and form resized.

```

1  1
2  2
3  3
4  4
5  5
6  6
7  7
8  8
9  9
10 namespace FileIOAndLINQ.Models
11 {
12     6 references
13     class VerseXlsxModel
14     {
15         // Class level properties
16         3 references
17         public double Id { get; set; }
18         3 references
19         public string Book { get; set; }
20         3 references
21         public double Chapter { get; set; }
22         3 references
23         public string Verse { get; set; }
24         3 references
25         public string Text { get; set; }
26         3 references
27         public string Meaning { get; set; }
28         3 references
29         public double Importance { get; set; }
30
31         /// <summary> Default constructor for the Verse Data Model
32         6 references
33         public VerseXlsxModel()
34         {
35             // Set the properties to default values
36             Id = 0;
37             Book = "";
38             Chapter = 0;
39             Verse = "";
40             Text = "";
41             Meaning = "";
42             Importance = 0;
43         }
44
45         /// <summary> Parameterized constructor for the Verse Request Model
46         6 references
47         public VerseXlsxModel(double id, string book, double chapter, string verse, string text, string meaning, double importance)
48         {
49             // Set the properties equal to the corresponding parameters
50             Id = id;
51             Book = book;
52             Chapter = chapter;
53             Verse = verse;
54             Text = text;
55             Meaning = meaning;
56             Importance = importance;
57         }
58
59         /// <summary>
60         /// <summary> Quick and dirty conversion method to change the xlsx model to a data model
61         /// </summary>
62         /// <returns></returns>
63
64         1 reference
65         public VerseDataModel ToDataModel()
66         {
67             // Create a new Verse Data Model from the xlsx model properties
68             VerseDataModel verseData = new VerseDataModel((int)Id, Book, (int)Chapter, Verse, Text, Meaning, (int)Importance);
69             // Return the now properly formatted Verse Data
70             return verseData;
71         }
72     }
73 }

```

Figure 4-3: Screenshot of the code behind the verse xlsx model I added in to receive the data coming in from the excel files. Excel stores numbers as doubles and the data model is int so I created a small class to hold the doubles and then create and return a data model with ToDataModel() method. I am not 100% sure putting the ToDataModel here was the best practice but it was quick and clean which I guess is the best practice.

```
VerseDAO.cs - X
FileDAOAndLINQ - %FileDAOAndLINQ\Services\DataAccessLayer\VerseDAO - WriteVersesToFile(string fileName)

96
97
98 case ".xml":
99     // Use ServiceStack.Text.XmlSerializer to serialize to xml
100     serialized = ServiceStack.Text.XmlSerializer.SerializeToString(verses);
101     break;
102
103 case ".xlsx":
104     // Open the EPPlus Excel Package for use with the user selected fileName
105     using (var package = new ExcelPackage(fileName))
106     {
107         // Initialize the variable sheet with the Bible Verses sheet if it exists.
108         var sheet = package.Workbook.Worksheets["Bible Verses"];
109
110         // If the Bible Verses sheet does exist we need to remove it.
111         if (sheet != null)
112         {
113             package.Workbook.Worksheets.Delete(sheet);
114         }
115
116         // Set the sheet as a new sheet
117         sheet = package.Workbook.Worksheets.Add("Bible Verses");
118
119         // Fill the sheet with our data into a table starting at A1
120         sheet.Cells["A1"].LoadFromCollection(verses, true, TableStyles.Medium);
121
122         // Format the newly added table
123         sheet.Columns[1].Width = 4;
124         sheet.Columns[1].Style.HorizontalAlignment = ExcelHorizontalAlignment.Center;
125         sheet.Columns[2].Width = 20;
126         sheet.Columns[2].Style.HorizontalAlignment = ExcelHorizontalAlignment.Left;
127         sheet.Columns[3].Width = 10;
128         sheet.Columns[3].Style.HorizontalAlignment = ExcelHorizontalAlignment.Center;
129         sheet.Columns[4].Width = 8;
130         sheet.Columns[4].Style.HorizontalAlignment = ExcelHorizontalAlignment.Center;
131         sheet.Columns[5].Width = 50;
132         sheet.Columns[5].Style.HorizontalAlignment = ExcelHorizontalAlignment.Justify;
133         sheet.Columns[6].Width = 50;
134         sheet.Columns[6].Style.HorizontalAlignment = ExcelHorizontalAlignment.Justify;
135         sheet.Columns[7].Width = 12.8;
136         sheet.Columns[7].Style.HorizontalAlignment = ExcelHorizontalAlignment.Center;
137
138         sheet.Rows.Height = 40;
139
140         // Save to file
141         package.Save();
142     }
143     return "The Excel file was successfully created";
144
145 default:
146     return "File not recognized";
147 }
148
149 // Try to save the serialized string to file.
150 try
151 {
152     // Use File.WriteAllText to send the serialized string to the file
153     File.WriteAllText(fileName, serialized);
154 }
155 catch (Exception ex)
156 {
157     return ex.Message;
158 }
159 // Return success message to the user
160 return "The verses have been saved to your file.";
161 }
```

Figure 4-4: Screenshot of the case additions to write verses to file for .xml and .xlsx. The xml addition went pretty smoothly; the Excel addition was a little more complicated and required a bit of trial and error with the documentation on the EPPlus web site to get the look I wanted in the outputted file. Another issue was figuring out how to handle file overwrites. I ended up needing to delete the worksheet every time and rewrite it.

```

VerseDAO.cs
FileDAndInQ
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248

case ".xml":
    // Deserialize the data using XmlSerializer
    dataVerses = ServiceStack.Text.XmlSerializer.DeserializeFromStrings<VerseDataModel>(data);
    break;

case ".xlsx":
    // Open the EPPlus Excel Package for use with the user selected filename
    using (var package = new ExcelPackage(fileName))
    {
        // Check if the worksheet exists
        var worksheet = package.Workbook.Worksheets["Bible Verses"];
        if (worksheet == null)
        {
            return "Worksheet 'Bible Verses' not found";
        }

        // Check if there's any data
        if (worksheet.Dimension == null || worksheet.Dimension.Rows < 2)
        {
            return "No data found in worksheet";
        }

        // Create a xlsx data model list to hold our incoming table.
        // Incoming data from Excel stores numbers as doubles. Creating this new model allows me to
        // easily convert the data coming in, into the data we need.
        List<VerseXlsxModel> xlsxVerses = new List<VerseXlsxModel>();
        // Fill the model list with the data in the xlsx file.
        xlsxVerses = worksheet.Tables[0].ToCollection<VerseXlsxModel>();

        // Convert each list item from xlsx data to verse data and add it to the dataVerses list.
        foreach (VerseXlsxModel verse in xlsxVerses)
        {
            // Convert and add one line.
            dataVerses.Add(verse.ToDataModel());
        }
    }
    break;

```

Figure 4-5: Screenshot of the case additions to read verses from file for .xml and .xlsx. Again, xml was really straight forward using the previously installed ServiceStack. At this point the reading of the Excel file was pretty painful. I tried several different fixes and methods for getting the Excel data back into the data grid view. After a few hours of messing around what turned out to be the issue was the numbers in Excel being represented as double values and the data model I was trying to store the files data in looking for int values. I created an additional class to hold the excel data and then converted it after the fact to a verse data model. This got everything working as intended.

```

VerseDAO.cs
FileDAndInQ
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390

/// <summary>
/// Public DAO method to count the number of verses in the list of verses entries.
/// </summary>
/// <returns></returns>
// ReSharper disable once SuggestVarOrTypeForUsedParameters
public int GetTotalNumberOfVerses()
{
    // Declare and Initialize
    int totalCount = 0;

    // Count each verse entry
    foreach (var verse in _verses)
    {
        // Add up the total verses
        totalCount += CountVersesInReference(verse.Verse);
    }

    // Return the final count
    return totalCount;
}

/// <summary>
/// Private method to count the number of verses in a single verse entry to handle spans (i.e. 1-6)
/// </summary>
/// <param name="verse"></param>
/// <returns></returns>
private int CountVersesInReference(string verse)
{
    // Catch null or empty strings
    if (string.IsNullOrEmpty(verse))
        return 0;

    try
    {
        // Check for range (handle both hyphen and on dash)
        // With Regex validating our input this is a bit of overkill
        // but better safe than sorry later
        if (verse.Contains("-") || verse.Contains("--"))
        {
            // Replace on dash with regular hyphen for consistency going forward
            verse = verse.Replace("--", "-");

            // Split the start and end values to a range array
            var verseRange = verse.Split("-");
            // If the array has exactly 2 elements (start and end)
            if (verseRange.Length == 2)
            {
                // Parse the start and end values
                if (int.TryParse(verseRange[0].Trim(), out int startVerse) &&
                    int.TryParse(verseRange[1].Trim(), out int endVerse))
                {
                    // Calculate the span (inclusive)
                    return (endVerse - startVerse) + 1;
                }
            }
        }

        // Single verse with no hyphen
        return 1;
    }
    catch
    {
        // If parsing fails, count as 1
        return 1;
    }
}

```

Figure 4-6: Screenshot of the get total number of verses and count verses in reference methods used to get the total number of verses the user has collected.

```

391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
FileDAO.Linq VerseDAO.cs %FileDAO.Linq.Services.DataAccessLayer.VerseDAO WriteVersesToFile(string fileName)
/// <summary>
/// Searches the verses for matching search terms and returns any matches.
/// </summary>
/// <param name="searchText"></param>
/// <returns></returns>
reference
public List<VerseDataModel> SearchVerses(string searchText)
{
    // Create a verse data model to hold the search results
    List<VerseDataModel> filteredVerses;

    // Check if the search term is blank
    if (string.IsNullOrWhiteSpace(searchText))
    {
        // Return all verses if search is empty
        filteredVerses = _verses;
    }
    else
    {
        // Convert the search term to lower case
        string searchLower = searchText.ToLower();

        // Use method syntax LINQ to filter the searched terms for text and meaning
        filteredVerses = _verses.Where(v =>
            (v.Text?.ToLower().Contains(searchLower) ?? false)
            || (v.Meaning?.ToLower().Contains(searchLower) ?? false))
            .ToList();

        // Return the search filtered verses
        return filteredVerses;
    }
}

```

Figure 4-7: Screenshot of the search verses method in the Verse DAO that finds and returns all verses that match the given search term. LINQ method syntax is used to iterate over all the `_verses` and add the ones that match to the filtered verses list before returning the filtered list.

```

130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
VerseLogic.cs VerseDAO.cs %FileDAO.Linq.Services.BusinessLogicLayer.VerseLogic ReadVerseFromFile(string fileName)
/// <summary>
/// Get the int value of the number of verses in the list and return it as a string
/// </summary>
/// <returns></returns>
reference
public string GetTotalNumberOfVerses()
{
    // Return DAO Method
    return _verseDAO.GetTotalNumberOfVerses().ToString();
}

/// <summary>
/// Get the list of verses that match the search term from the DAO
/// then convert it to a display model and return it.
/// </summary>
/// <param name="searchText"></param>
/// <returns></returns>
reference
public List<VerseDisplayModel> SearchVerses(string searchText)
{
    // Get the search results from the DAO
    List<VerseDataModel> dataVerses = _verseDAO.SearchVerses(searchText);
    // Convert the returned data verses to display and return them
    return ConvertVerseDataToDisplay(dataVerses);
}

```

Figure 4-8: Screenshot of the changes to the verse logic class. The read and write file methods were unchanged, all the heavy lifting there was done in the DAO. I did add some methods to get the number of verses and the search results from the DAO.


```
frmVersesList.cs [Design]
File\OAndU\NQ.Presentation\Layer\FrmVersesList
//errorLabels

539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587

/// <summary>
/// Search button click event handler
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
/// </param>
/// </summary>
private void BtnSearchClickEH(object sender, EventArgs e)
{
    // Get the search text from the textbox
    string searchText = txtSearch.Text;

    // Get the filtered list of verses from the BLL
    List<VerseDisplayModel> searchResults = _verseLogic.SearchVerses(searchText);

    // Change the datasource for the verse binding source
    _verseBindingSource.DataSource = searchResults;

    // Format the data grid view
    FormatVersesDgv();
}

/// <summary>
/// Search text box key down event handler to catch hitting enter after typing
/// to simulate the search button click.
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
/// </param>
/// </summary>
private void TxtSearchKeyDownEH(object sender, KeyEventArgs e)
{
    // did the user hit enter?
    if (e.KeyCode == Keys.Enter)
    {
        // Prevent the 'ding' sound
        e.SuppressKeyPress = true;

        // Call the search button click event handler
        BtnSearchClickEH(sender, e);
    }
}

/// <summary>
/// Click event handler for the exit menu item.
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
/// </param>
/// </summary>
private void TsmExitClickEH(object sender, EventArgs e)
{
    this.Close();
}
```

Figure 4-9: Screenshot of event handlers added to the form verse list to handle the presentation of the search functionality and the final menu item Exit implemented.

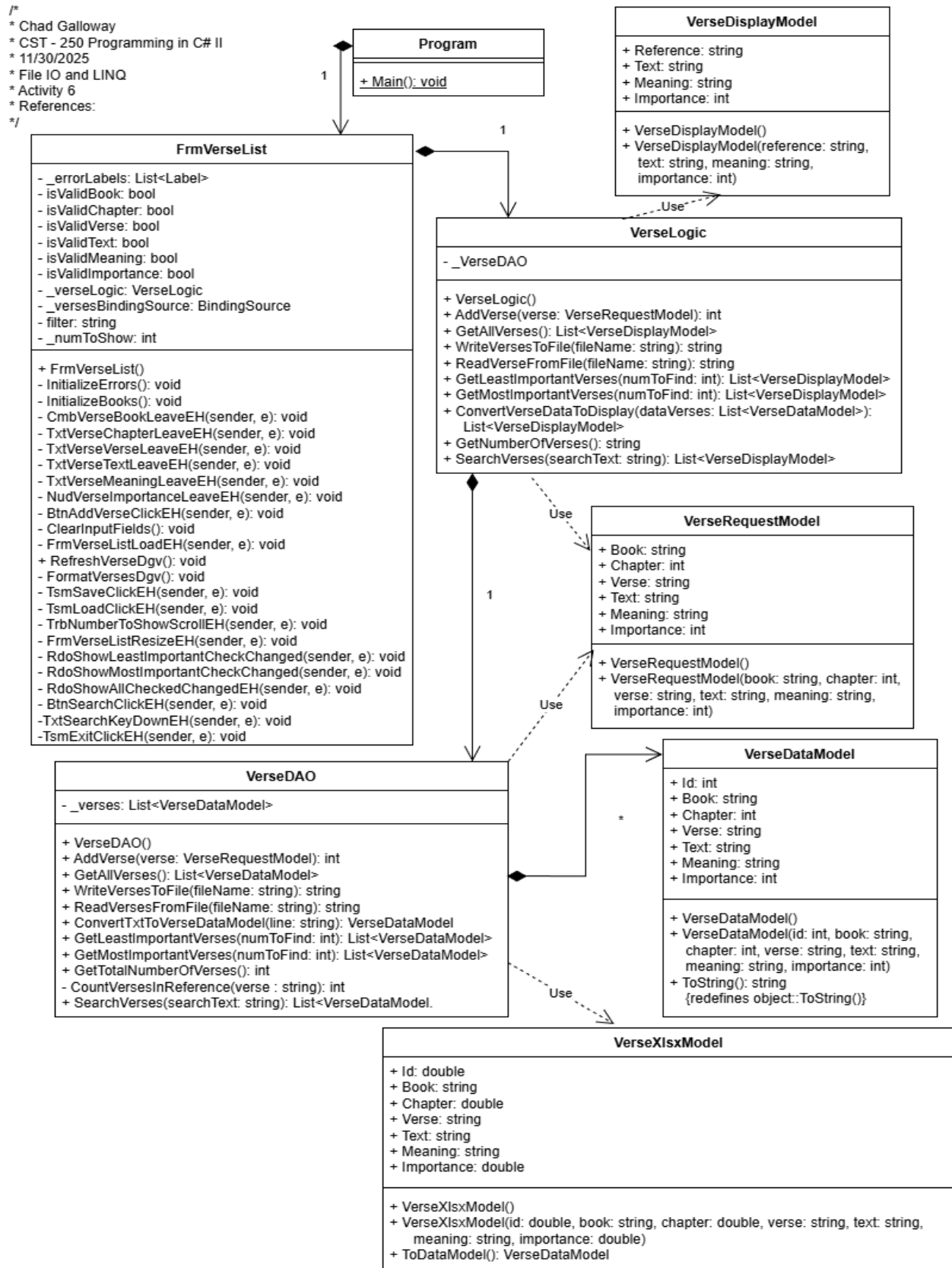


Figure 4-10: Screenshot of the UML as it stands now.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<ArrayOfVerseDataModel xmlns:i="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://schemas.datacontract.org/2004/07/FileIOAndLINQ.Models">
  <VerseDataModel>
    <Book>John</Book>
    <Chapter>8</Chapter>
    <Id>1</Id>
    <Importance>7</Importance>
    <Meaning>The meaning is that a spiritual, experiential knowledge of God's truth, particularly through following Jesus, leads to freedom from the bondage of sin, guilt, and deception.</Meaning>
    <Text>Then you will know the truth, and the truth will set you free.</Text>
    <Verse>32</Verse>
  </VerseDataModel>
  <VerseDataModel>
    <Book>Proverbs</Book>
    <Chapter>22</Chapter>
    <Id>2</Id>
    <Importance>4</Importance>
    <Meaning>Parents who teach their children to love God and respect his laws can expect such training to have a positive and lasting influence.</Meaning>
    <Text>Train a boy in the way he should go; even when he grows old he will not depart from it.</Text>
    <Verse>6</Verse>
  </VerseDataModel>
  <VerseDataModel>
    <Book>Isaiah</Book>
    <Chapter>42</Chapter>
    <Id>3</Id>
    <Importance>2</Importance>
    <Meaning>God tells us his personal name and adds that he does not share his praise or honor with idols.</Meaning>
    <Text>I am Jehovah. That is my name; I give my glory to no one else, nor my praise to graven images.</Text>
    <Verse>8</Verse>
  </VerseDataModel>
  <VerseDataModel>
    <Book>John</Book>
    <Chapter>3</Chapter>
    <Id>4</Id>
    <Importance>9</Importance>
    <Meaning>God loves us and wants us to live forever. For that reason, he sent his Son, Jesus Christ, to the earth. While on earth, Jesus accomplished much. For one thing, he taught his followers about his God and Father. (1 Peter 1:3) For another, he gave his life for mankind. To gain everlasting life, we must have faith in Jesus.</Meaning>
    <Text>For God loved the world so much that he gave his only-begotten Son, so that everyone exercising faith in him might not be destroyed but have everlasting life.</Text>
    <Verse>16</Verse>
  </VerseDataModel>
  <VerseDataModel>
    <Book>2 Corinthians</Book>
    <Chapter>4</Chapter>
    <Id>5</Id>
    <Importance>3</Importance>
    <Meaning>The meaning is to shift focus from the temporary, physical, and seen to the eternal, spiritual, and unseen, which provides hope and strength for daily renewal. </Meaning>
    <Text>So we do not lose heart. Though our outer self is wasting away, our inner self is being renewed day by day. For this light momentary affliction is preparing for us an eternal weight of glory beyond all comparison, as we look not to the things that are seen but to the things that are unseen.</Text>
    <Verse>16-18</Verse>
  </VerseDataModel>
</ArrayOfVerseDataModel>
```

Figure 4-11: Screenshot of xml file opened in Microsoft Edge

	Book	Chapter	Vers	Text	Meaning	Importance
1					of God's truth, particularly through following Jesus, leads to freedom from the bondage of sin, guilt, and deception.	7
2	1 John	8	32	Then you will know the truth, and the truth will set you free.		
3	2 Proverbs	22	6	Train a boy in the way he should go; even when he grows old he will not depart from it.	Parents who teach their children to love God and respect his laws can expect such training to have a positive and lasting influence.	4
4	3 Isaiah	42	8	I am Jehovah. That is my name; I give my glory to no one else, nor my praise to graven images.	God tells us his personal name and adds that he does not share his praise or honor with idols.	2
5	4 John	3	16	For God loved the world so much that he gave his only-begotten Son, so that everyone exercising faith in him might not be destroyed but have everlasting life.	reason, he sent his Son, Jesus Christ, to the earth. While on earth, Jesus accomplished much. For one thing, he taught his followers about his God and Father. (1 Peter	9
6	5 2 Corinthians	4	16-18	away, our inner self is being renewed day by day. For this light momentary affliction is preparing for us an eternal weight of glory beyond all comparison, as we	The meaning is to shift focus from the temporary, an physical, and seen to the eternal, spiritual, and unseen, which provides hope and strength for daily renewal.	3
7						
8						
9						

Figure 4-12: Screenshot of the xlsx file opened in Excel.